

Capitolo 17 — PROC-001 — Service Job Lifecycle

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-17
Data master	2026-08-29
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/17_proc_001_service_job_lifecycle.md
Source SHA	16b4765
Release	2026-08-29T15:44:27.282Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Parte VI — Il Libro dei Processi WCM Stato: FROZEN **Data:** 2026-08-29 **Scope:** WCM generale, domain-agnostic

17.0 Dal processo astratto alla prima unità di lavoro

Nel capitolo precedente abbiamo costruito una grammatica per leggere i processi WCM. Ora la applichiamo al primo processo del Process Book:

PROC-001 — SERVICE JOB LIFECYCLE

Il suo problema sembra semplice:

Come facciamo a sapere se un lavoro delegato può partire, è davvero in esecuzione, è bloccato oppure è realmente concluso?

In un sistema tradizionale potremmo accontentarci di una lista di attività con etichette come “da fare”, “in corso” e “fatto”.

In un sistema che combina persone, AI, automazioni, repository, servizi e authority, non basta.

Un'etichetta può essere aggiornata troppo presto.

Un agente può essere svegliato due volte sullo stesso lavoro.

Una sessione può terminare mentre il lavoro è ancora vivo.

Un servizio può dichiarare di avere finito senza che l'output sia stato verificato.

Un nuovo runtime può non ricordare che il job era già stato preso in carico.

PROC-001 esiste per impedire che queste ambiguità diventino stato operativo.

Il suo principio di fondo è:

IL LAVORO DEVE AVERE
UNO STATO PERSISTENTE,
VERIFICABILE
E IDEMPOTENTE.

17.1 Che cos'è un Service Job

Un **Service Job** è un'unità di lavoro delegata e governata.

Non coincide necessariamente con un intero progetto.

Non coincide nemmeno con una singola chiamata a un modello AI.

È un contratto operativo sufficientemente definito da permettere a un esecutore autorizzato di capire almeno:

- che cosa deve fare;
- entro quale perimetro;
- con quali input;
- su quali risorse;
- con quali limiti;
- quali output deve produrre;
- come verrà verificato il risultato.

Possiamo pensarlo così:

```
SERVICE JOB
=
UNITÀ DI LAVORO
+ SCOPE
+ AUTHORITY
+ STATO
+ CRITERI DI ACCETTAZIONE
```

Il lifecycle non governa quindi solo *quando* un lavoro viene eseguito.

Governa anche **quando è lecito considerarlo eseguibile, preso in carico e concluso**.

17.2 Perché il lifecycle deve essere persistente

Immaginiamo che un agente riceva un compito e cominci a lavorare.

Durante l'esecuzione la sessione termina.

Un'ora dopo un altro agente o un altro heartbeat osserva il sistema.

Se lo stato del lavoro esisteva soltanto nella memoria della sessione precedente, il nuovo esecutore potrebbe concludere che:

```
"non so cosa stesse succedendo"
```

oppure, peggio:

```
"sembra ancora da fare"
```

e ripetere il lavoro.

PROC-001 evita che il significato operativo dipenda dalla memoria volatile del runtime.

La baseline stabilisce infatti che:

```
PERSISTENT JOB STATE
>
VOLATILE RUNTIME MEMORY
```

Questo non significa che la memoria viva non serva.

Significa che, quando dobbiamo sapere **qual è lo stato effettivo del job**, la fonte persistente del Service Job prevale sul ricordo della sessione.

17.3 Il lifecycle standard

Il percorso principale è:

```
HOLD
↓
AUTHORIZATION / ACTIVATION
↓
READY
↓
DURABLE DISPATCH + VALID CLAIM
↓
IN_PROGRESS
↓
VERIFIED ACCEPTANCE
↓
DONE
```

HOLD è opzionale.

Le transizioni centrali sono quindi:

```
READY
→
IN_PROGRESS
→
DONE
```

Ma il valore del processo non sta nelle tre etichette.

Sta nelle **condizioni che rendono legittimo passare da uno stato al successivo**.

17.4 HOLD — definito non significa eseguibile

HOLD rappresenta un job che esiste, ma che non è ancora eseguibile.

Può essere già descritto.

Può avere un obiettivo plausibile.

Può perfino avere un esecutore potenziale.

Ma manca ancora qualcosa che autorizzi l'avvio.

La regola è netta:

```
HOLD
```

```
≠  
READY
```

e soprattutto:

```
HOLD  
=  
NON ESEGUIBILE
```

Questa distinzione evita un errore molto comune nei sistemi agentici: confondere la presenza di un task con il permesso di eseguirlo.

Un lavoro può essere noto al sistema senza essere ancora attivabile.

17.5 READY — il lavoro è autorizzato nel perimetro definito

READY significa che il job può essere eseguito nel proprio scope.

Non significa:

- libertà di ridefinire il goal;
- libertà di allargare il budget;
- libertà di modificare qualsiasi path;
- libertà di cambiare branch;
- libertà di superare gate di governance;
- libertà di inventare nuovi permessi.

L'esecutore riceve authority **sul lavoro definito**, non authority generale.

Possiamo leggere **READY** così:

```
READY  
=  
ESEGUIBILE  
ENTRO IL CONTRATTO
```

Il runtime non può usare l'esistenza del job per ampliare autonomamente:

```
goal  
scope  
budget  
permissions  
authority
```

Questa è una delle ragioni per cui il Service Job funziona come contratto operativo e non come semplice promemoria.

17.6 READY non significa «sveglia un agente quante volte vuoi»

Qui entra una distinzione fondamentale.

Un job **READY** è eleggibile per l'esecuzione.

Ma questo non autorizza il control plane a generare più esecuzioni cognitive equivalenti.

Se lo stesso job, sulla stessa versione, viene osservato da tre tick successivi, non devono nascere tre copie dello stesso lavoro.

Quando il job viene attivato attraverso un control plane periodico, la baseline collega PROC-001 a:

```
PROC-003
+
PROT-004
```

Il principio diventa:

```
UN LAVORO
→
UN DISPATCH LOGICO
→
UN CLAIM VALIDO
```

Non:

```
UN JOB READY
→
N WAKE
→
N ESECUZIONI DUPLICATE
```

Questa è la dimensione idempotente del lifecycle.

17.7 Il durable dispatch: trasportare il lavoro senza cambiare la source of truth

Quando il runtime richiede un envelope persistente per consegnare il lavoro a un esecutore cognitivo, entra in gioco il **durable canonical dispatch**.

Il flusso tipico è:

```
READY
↓
DURABLE DISPATCH
↓
ASSIGNMENT / CLAIM
↓
EXECUTION
```

Il dispatch può trasportare o rendere risolvibili elementi come:

- Service Job ID;
- path;
- progetto o scope operativo;
- branch;

- versione remota osservata;
- assignee autorizzato.

Ma c'è una distinzione da proteggere:

```
SERVICE JOB
=
CONTRATTO + STATO DI VERITÀ

DISPATCH
=
ENVELOPE + CLAIM + AUDIT
```

Il dispatch non sostituisce il Service Job.

Non acquisisce authority aggiuntiva.

Non diventa automaticamente la nuova source of truth del lifecycle.

Questa separazione permette di cambiare il meccanismo di trasporto senza cambiare il significato del lavoro.

17.8 Il claim: il punto in cui l'eleggibilità viene consumata

Il **claim** è il momento in cui un esecutore autorizzato prende realmente in carico il lavoro.

È una transizione importante perché modifica il significato di **READY**.

Prima del claim il job è eleggibile.

Dopo un claim valido, i loop successivi non devono trattarlo come nuovo lavoro equivalente da rispedire.

La baseline esprime questo principio così:

```
VALID CLAIM
→
CONSUME OPERATIONAL ELIGIBILITY
```

In termini pratici:

```
READY
→ claim valido
→ nessun secondo claim equivalente
```

L'idempotenza non è quindi soltanto una protezione tecnica contro richieste duplicate.

È una proprietà del lifecycle.

17.9 Prima di IN_PROGRESS: il Pre-Execution Gate

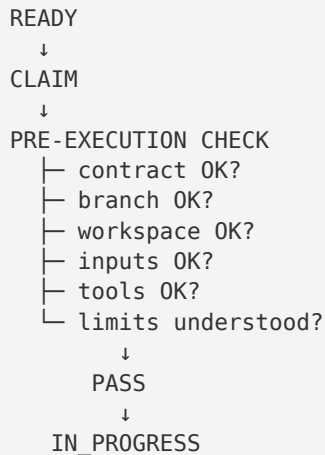
PROC-001 non permette di passare a **IN_PROGRESS** soltanto perché un agente è stato svegliato.

Prima devono essere verificati gli elementi operativi necessari.

La baseline richiede almeno:

- contratto del job;
- branch;
- workspace;
- input;
- strumenti;
- limiti.

Possiamo rappresentare il gate così:



Se il gate non passa, non dobbiamo fingere che il lavoro sia realmente in esecuzione.

La classificazione corretta dipenderà dalla causa.

Ed è qui che diventano importanti gli stati alternativi.

17.10 IN_PROGRESS — lavoro realmente preso in carico

IN_PROGRESS non dovrebbe significare:

❌ *“qualcuno ha visto il task”.*

Dovrebbe significare:

❌ **un esecutore valido ha preso in carico il Service Job e le condizioni operative necessarie all'esecuzione sono state verificate.**

Questa differenza rende lo stato informativo.

In un sistema distribuito, ogni stato dovrebbe permettere a un osservatore successivo di capire che cosa è già successo senza doverlo indovinare.

Per questo:

```
IN_PROGRESS
≠
WAKE AVVENUTO
```

e:

```
IN_PROGRESS
≠
INTENZIONE DI LAVORARE
```

È uno stato operativo persistente.

17.11 DONE — la parola più pericolosa

DONE sembra lo stato più semplice.

In realtà è quello che richiede più disciplina.

PROC-001 non consente:

```
"ho prodotto qualcosa"
→ DONE
```

né:

```
"il servizio dice di aver finito"
→ DONE
```

La transizione corretta è:

```
OUTPUT
↓
VERIFICA
↓
ACCEPTANCE CRITERIA
↓
EVIDENCE
↓
DONE
```

Qui PROC-001 lavora insieme a:

```
PROT-002 — Result Acceptance & Closure
```

La regola fondamentale di PROT-002 è:

```
DONE
RICHIEDE
EVIDENZA VERIFICABILE
```

17.12 Che cosa significa verificare l'acceptance

Prima di dichiarare **DONE**, il sistema deve verificare gli acceptance criteria applicabili.

La baseline richiede almeno di controllare:

- che gli output previsti esistano;
- che il loro contenuto sia coerente con il job;

- che le modifiche siano rimaste nei path autorizzati;
- che branch e destinazione siano corretti quando pertinenti;
- che test ed evidenze significative siano registrati;
- che eventuali limiti siano dichiarati;
- che un risultato parziale non venga presentato come successo completo.

Questo è un principio molto più forte di:

```
TASK EXECUTED
```

La vera domanda è:

```
RESULT ACCEPTED?
```

Solo dopo possiamo passare a **DONE**.

17.13 Verifica indipendente quando possibile

Un altro principio importante riguarda chi verifica.

Se l'orchestratore dispone di accesso diretto alla source of truth, dovrebbe preferire la verifica reale rispetto alla sola dichiarazione del service.

Il pattern è:

```
SERVICE DICHIARA DONE
  ↓
VERIFICA DIRETTA OUTPUT / STATO
  ↓
COERENTE?
├─ SÌ → ACCEPT
└─ NO → INDAGA / ESCALA
```

Questo riduce una forma sottile di non determinismo organizzativo:

prendere come verità una frase prodotta dall'esecutore senza controllare il risultato persistente.

Il punto non è “non fidarsi dell'AI”.

Il punto è applicare la stessa disciplina che useremmo in qualsiasi sistema controllato:

```
CLAIM
≠
EVIDENCE
```

17.14 Gli stati alternativi

Non tutti i job arrivano direttamente da **IN_PROGRESS** a **DONE**.

PROC-001 definisce quattro stati alternativi principali:

```
BLOCKED_LOCAL
BLOCKED_WISE
```

FAILED
CANCELLED

Non sono sinonimi.

Servono a distinguere cause operative molto diverse.

17.15 BLOCKED_LOCAL — il problema è interno al perimetro operativo

BLOCKED_LOCAL indica un impedimento temporaneo che può essere gestito dal service nel proprio scope.

Per esempio, in astratto:

- un file locale non è ancora disponibile ma può essere rigenerato;
- una verifica tecnica deve essere ripetuta;
- una dipendenza operativa interna è momentaneamente indisponibile;
- serve una correzione che non modifica goal, authority o contratto.

Il punto fondamentale è:

BLOCKED_LOCAL
≠
SERVE UNA NUOVA DECISIONE DI GOVERNANCE

Il service può gestire il problema senza trasformarlo artificialmente in un gate umano.

17.16 BLOCKED_WISE — serve una decisione o un'autorità superiore

BLOCKED_WISE è diverso.

Qui manca qualcosa che l'esecutore non è autorizzato a inventare.

Può servire:

- una decisione;
- un'autorizzazione;
- un'informazione autorevole;
- una modifica del contratto;
- un chiarimento di scope;
- un'azione che supera i permessi disponibili.

La domanda diagnostica è:

Per proseguire devo cambiare che cosa stiamo facendo, perché lo stiamo facendo o con quale authority?

Se sì, il problema non è semplicemente locale.

Il lifecycle deve mostrare che il job attende un intervento superiore.

17.17 FAILED — il job non è completabile nelle condizioni correnti

FAILED indica che il lavoro non può essere portato a compimento nelle condizioni attuali. È diverso da un blocco temporaneo.

Un failure terminale deve rendere osservabile almeno:

- dove si è verificato il problema;
- quale condizione non è stata soddisfatta;
- quali output sono eventualmente parziali;
- che cosa impedisce la closure positiva.

Anche qui vale una regola importante:

```
FAILED
≠
DONE CON NOTE
```

Un sistema maturo non nasconde un fallimento dentro una chiusura apparentemente positiva.

17.18 CANCELLED — il lavoro viene terminato per authority

CANCELLED indica che il job viene terminato da authority competente.

Non è un failure tecnico.

Non significa che l'esecutore non fosse capace di completarlo.

Significa che il lavoro non deve più proseguire.

Possiamo distinguere:

```
FAILED
=
NON POSSIAMO COMPLETARE

CANCELLED
=
NON DOBBIAMO PIÙ COMPLETARE
```

Sono due fatti organizzativi diversi e devono rimanere distinguibili.

17.19 Un job concluso non torna automaticamente eleggibile

Una volta raggiunto **DONE**, il Service Job non deve essere rieseguito automaticamente.

La baseline stabilisce:

```
DONE
→
NO RE-RUN
```

salvo riapertura esplicita.

Questo protegge il sistema da una forma frequente di duplicazione:

1. il job viene completato;
2. un nuovo loop vede ancora lo stesso oggetto;
3. il runtime dimentica che era terminale;
4. il lavoro ricomincia.

La persistenza dello stato e la terminalità impediscono questa regressione.

17.20 Il closure packet: lasciare dietro di sé una storia verificabile

La chiusura minima di PROC-001 non è soltanto una parola di stato.

Deve permettere a un osservatore successivo di capire che cosa è successo senza ricostruire tutta la sessione.

La baseline richiede che la closure renda disponibili, quando applicabili:

- stato finale;
- sintesi del lavoro;
- output e path;
- evidence e test;
- acceptance criteria passati o falliti;
- problemi o limiti residui;
- azioni sensibili non eseguite;
- eventuale impatto sulla Knowledge Base;
- eventuale capability gap;
- prossimo passo raccomandato.

Se esiste un durable dispatch, anche la sua disposizione deve essere coerente con il risultato del Service Job.

In forma compatta:

```
DONE
=
STATO TERMINALE
+ RISULTATO
+ VERIFICA
+ LINEAGE
```

17.21 Service Job e dispatch devono chiudersi in modo coerente

Poiché il Service Job e il durable dispatch sono oggetti differenti, possono teoricamente divergere.

Per esempio:

```
SERVICE JOB = DONE  
DISPATCH = IN_PROGRESS
```

oppure:

```
SERVICE JOB = FAILED  
DISPATCH = DONE
```

Queste combinazioni renderebbero ambiguo il sistema.

La baseline richiede quindi coerenza tra il risultato del job e la disposizione del dispatch.

Il principio è:

```
TRANSPORT STATE  
DEVE ESSERE COERENTE CON  
WORK STATE
```

senza confondere i due livelli.

17.22 Dove entra il determinismo

PROC-001 contiene sia comprensione semantica sia vincoli che possono essere verificati meccanicamente.

È cognitivo capire, per esempio:

- se un acceptance criterion semantico è davvero soddisfatto;
- se un limite residuo cambia il significato del risultato;
- se una richiesta implica un'estensione non autorizzata dello scope.

È invece fortemente deterministico verificare, quando il contratto lo permette:

- lo stato corrente del job;
- la presenza di un output;
- il branch;
- il path;
- l'esistenza di un dispatch equivalente;
- una chiave di idempotenza;
- la terminalità di **DONE**.

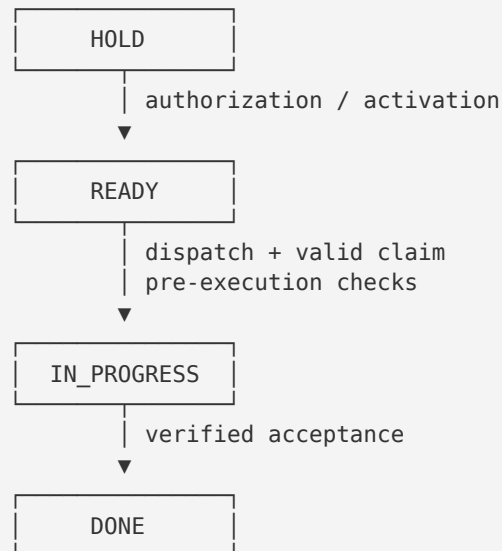
Questa separazione è importante.

Il processo non tenta di rendere “deterministico” ciò che richiede comprensione.

Tenta di rendere **deterministiche le proprietà che possono esserlo**, lasciando il reasoning dove serve.

17.23 Il lifecycle visto come macchina a stati

Possiamo riassumere PROC-001 come una macchina a stati governata:



Da READY / IN_PROGRESS possono emergere:
BLOCKED_LOCAL · BLOCKED_WISE · FAILED · CANCELLED

Questa figura testuale è sufficiente per il capitolo.

Una grafica separata aggiungerebbe poco alla comprensione e non è necessaria.

17.24 Un esempio astratto

Immaginiamo un Service Job che richiede di produrre un report a partire da un set di dati già disponibile.

Il job viene definito, ma non è ancora autorizzato.

HOLD

Arriva l'attivazione prevista.

Scope, input e acceptance criteria sono ora sufficientemente definiti.

READY

Il control plane verifica che non esista già un dispatch equivalente e materializza un solo envelope durevole.

L'esecutore autorizzato prende in carico il job.

Contratto, workspace, input, strumenti e limiti passano il pre-execution check.

IN_PROGRESS

L'esecutore produce il report.

Ma il processo non passa ancora a **DONE**.

Prima vengono verificati:

- esistenza del file;
- contenuto richiesto;
- destinazione corretta;
- criteri di accettazione;
- limiti residui.

Solo dopo:

```
DONE
```

Se invece manca un dato che il service può recuperare nel proprio scope:

```
BLOCKED_LOCAL
```

Se serve cambiare l'obiettivo del report:

```
BLOCKED_WISE
```

Se il dataset è irrimediabilmente corrotto nelle condizioni correnti:

```
FAILED
```

Se l'authority decide che il report non serve più:

```
CANCELLED
```

Lo stesso vocabolario separa situazioni che, senza lifecycle, apparirebbero tutte semplicemente come “non finito”.

17.25 I failure mode che PROC-001 cerca di impedire

PROC-001 è più facile da capire se lo leggiamo attraverso gli errori che vuole rendere improbabili.

Failure mode 1 — Authority ambigua

Il job esiste, quindi l'esecutore presume di poter fare qualsiasi cosa necessaria.

Correzione:

```
READY  
=  
AUTHORITY NEL PERIMETRO DEFINITO
```

Failure mode 2 — Branch, path o workspace non autorizzati

Il lavoro parte nel posto sbagliato.

Correzione: Pre-Execution Gate prima di **IN_PROGRESS**.

Failure mode 3 — Input mancanti

L'agente comincia comunque e colma le lacune inventando.

Correzione: verificare input e classificare correttamente l'eventuale blocker.

Failure mode 4 — Acceptance non verificabile

Il servizio produce un output ma nessuno può dire se sia corretto.

Correzione: niente **DONE** senza acceptance verificabile.

Failure mode 5 — Job già terminale o già in esecuzione

Un nuovo loop ripete il lavoro.

Correzione: stato persistente + claim valido + idempotenza.

Failure mode 6 — Dispatch duplicati

La stessa unità di lavoro genera più run cognitive equivalenti.

Correzione: durable dispatch deduplicato per job/versione quando quel meccanismo è applicabile.

Failure mode 7 — Job lasciato READY dopo la presa in carico

Il control plane continua a considerarlo nuovo lavoro.

Correzione: claim e transizione persistente coerente.

Failure mode 8 — Serve una decisione che cambi il contratto

Il service continua autonomamente.

Correzione: **BLOCKED_WISE**, non espansione silenziosa dello scope.

17.26 Le relazioni principali

PROC-001 è il primo processo del Process Book, ma non lavora isolato.

Le relazioni più immediate sono:

```
PROC-001
├─ USES → PROT-002 Result Acceptance & Closure
├─ WHEN PERIODIC DISPATCH APPLIES
└─ COOPERATES WITH → PROC-003 Deterministic Discovery & Durable Dispatch
```


| └ GUARDED BY → PROT-004 Canonical Dispatch & Idempotency
└ REQUIRES → verifica operativa di branch / workspace / input / tools / limits

Nel capitolo successivo entreremo in PROC-002, che affronta specificamente il problema dell'allineamento sicuro del workspace.

Qui è sufficiente capire che PROC-001 non consente di dichiarare **IN_PROGRESS** senza avere prima verificato il contesto operativo necessario.

17.27 Evidence e maturità

Nel Process Register, PROC-001 è classificato:

VALIDATED

La sua evidence include POC sul Service Bridge, esecuzioni zero-touch/routine-driven e prove successive sul rapporto tra lifecycle, dispatch e idempotenza.

Questa evidenza sostiene la baseline corrente.

Non significa però:

VALIDATED
=
DIMOSTRATO UNIVERSALMENTE
IN QUALSIASI ORGANIZZAZIONE

La qualifica va letta nello scope WCM corrente.

Il processo è parte della baseline operativa e ha evidenza concreta alle spalle, ma il libro non trasforma questa maturità in una pretesa di validità universale.

17.28 Il significato sistemico di PROC-001

PROC-001 rende possibile una proprietà fondamentale del WCM:

un lavoro delegato continua ad avere identità e stato anche quando cambiano sessione, runtime o esecutore.

Senza questa proprietà, un sistema agentic rischia di essere una successione di conversazioni.

Con un lifecycle persistente, invece, il lavoro diventa un oggetto organizzativo.

Possiamo vedere la trasformazione così:

PROMPT
→
INTENZIONE TRANSITORIA

SERVICE JOB
→
UNITÀ DI LAVORO GOVERNATA

SERVICE JOB + LIFECYCLE

→
UNITÀ DI LAVORO PERSISTENTE,
VERIFICABILE E RIPRENDIBILE

Questo è il motivo per cui PROC-001 viene prima degli altri processi operativi.

Prima di discutere come sincronizzare un workspace, scoprire lavoro, promuovere evidence o consolidare memoria, dobbiamo sapere **che cosa significa che un lavoro esiste, parte, viene preso in carico e finisce**.

17.29 La formula compatta

Possiamo comprimere l'intero processo in una formula:

```
DEFINED
≠
AUTHORIZED

AUTHORIZED
≠
CLAIMED

CLAIMED
≠
ACCEPTED

OUTPUT
≠
DONE

DONE
=
PERSISTENT STATE
+ VERIFIED ACCEPTANCE
+ EVIDENCE
+ CLOSURE
```

E possiamo aggiungere la regola di idempotenza:

```
SAME JOB + SAME VERSION
→
NO DUPLICATE EQUIVALENT EXECUTION
```

Queste due idee — **closure verificata** e **non duplicazione del lavoro** — sono il cuore di PROC-001.

17.30 Cosa abbiamo ottenuto

Ora il primo processo WCM non è più un diagramma di stati.

È una disciplina completa per governare un'unità di lavoro delegata.

Sappiamo che:

- **HOLD** esiste ma non è eseguibile;

- **READY** autorizza il lavoro nel perimetro definito;
- il durable dispatch, quando necessario, trasporta il lavoro senza sostituire la source of truth;
- un claim valido consuma l'eleggibilità operativa;
- **IN_PROGRESS** richiede un vero pre-execution check;
- **DONE** richiede acceptance verificata;
- **BLOCKED_LOCAL**, **BLOCKED_WISE**, **FAILED** e **CANCELLED** descrivono cause diverse;
- lo stato persistente prevale sulla memoria volatile;
- il runtime non può ampliare autonomamente l'authority;
- la closure deve lasciare evidence sufficiente a un osservatore successivo.

Nel prossimo capitolo entreremo in:

PROC-002 — Workspace Pre-Sync

Vedremo perché un esecutore non dovrebbe cominciare a modificare un workspace prima di sapere se il proprio stato locale è coerente con la baseline remota e come WCM riduce il rischio di lavorare su una realtà già superata.

Source Map

Fonti canoniche principali

- [wcm/process-book/processes/PROC-001_SERVICE_JOB_LIFECYCLE.md](#) — lifecycle, stati, gate, regole, closure, failure mode ed evidence;
- [wcm/process-book/protocols/PROT-002_RESULT_ACCEPTANCE_CLOSURE.md](#) — verifica degli acceptance criteria e closure evidence-based;
- [wcm/process-book/protocols/PROT-004_CANONICAL_DISPATCH_IDEMPOTENCY.md](#) — durable dispatch, claim, deduplicazione persistente e separazione dispatch/source of truth;
- [wcm/process-book/processes/PROC-003_DETERMINISTIC_DISCOVERY_DURABLE_DISPATCH.md](#) — relazione tra discovery deterministica, dispatch e lifecycle quando il job è attivato da control plane periodico;
- [wcm/process-book/PROCESS_REGISTER.md](#) — stato **VALIDATED**, scopo sintetico e posizione di PROC-001 nella baseline corrente;
- [WCM_AGENT_START.md](#) — source precedence, authority, persistent state e invarianti generali di esecuzione.

Relazioni

```
CH17
├ CONTINUES → CH16 – Come leggere un processo WCM
├ EXPLAINS → PROC-001 – Service Job Lifecycle
├ USES → PROT-002 – Result Acceptance & Closure
├ WHEN DISPATCH APPLIES
│ └ RELATES_TO → PROC-003 – Deterministic Discovery & Durable Dispatch
└ GUARDED_BY → PROT-004 – Canonical Dispatch & Idempotency
```

Maturity note

PROC-001 è **VALIDATED** nella baseline WCM corrente. La qualifica deriva dalle evidenze e dalle esecuzioni richiamate nel master tecnico e nel Process Register. Nel libro viene mantenuto il significato locale della maturità: **VALIDATED** non equivale a una dimostrazione universale in ogni dominio o organizzazione. Il capitolo è una spiegazione editoriale del processo corrente e non introduce nuove regole WCM.